

## **Integração e Entrega Continua**

### **Atividade Prática - Revisão para a Prova 2**

Aluno: Gabriel Camargo Prado

## **PARTE I - QUESTOES DE MULTIPLA ESCOLHA**

### **Questão 1 - Resposta: C**

Uma cobertura de apenas 38% significa que a maioria do código não está sendo executada pelos testes, ou seja, uma parcela significativa pode conter erros não detectados.

### **Questão 2 - Resposta: B**

O objetivo central dos testes automatizados em um pipeline CI/CD é detectar problemas antes da implantação, evitando que bugs cheguem ao ambiente de produção.

### **Questão 3 - Resposta: C**

O comando `npm audit` analisa as dependências do projeto em busca de vulnerabilidades conhecidas registradas em bases de dados públicas de segurança.

### **Questão 4 - Resposta: B**

Uma cobertura de 22% é muito baixa, indicando que a grande maioria do código não é exercitada pelos testes, e, portanto, a cobertura precisa ser ampliada.

### **Questão 5 - Resposta: C**

Os badges fornecem indicadores visuais e rápidos sobre a saúde do projeto (build, cobertura, vulnerabilidades), facilitando a leitura do estado atual por toda a equipe.

## **PARTE II - QUESTOES DISCURSIVAS**

### **Questão 6**

Teste automatizado é um script que verifica, de forma automática, se uma parte do código funciona conforme o esperado, sem intervenção manual. No projeto INPE, um exemplo seria um teste que verifica se o sistema envia corretamente um alerta de queimada ao detectar um foco de calor.

Cobertura de testes é a métrica que indica qual percentual do código-fonte é efetivamente executado durante os testes. No INPE, se o módulo de enchentes não tiver testes, ele não aparecerá na cobertura, mesmo que o restante do sistema esteja bem testado.

Vulnerabilidade de software é uma falha ou fraqueza em uma biblioteca ou componente que pode ser explorada por agentes maliciosos. No INPE, uma dependência desatualizada usada para processar imagens de satélite poderia conter uma vulnerabilidade que permitisse injeção de dados falsos nos alertas.

### **Questão 7**

Mesmo com 300 testes passando, uma cobertura de 40% significa que 60% do código nunca é executado durante os testes. No projeto INPE, isso representa um risco grave: funcionalidades críticas como o envio de alertas de enchentes ou queimadas podem conter bugs que jamais serão detectados antes de chegarem à produção. Testes que passam não garantem qualidade se não exercitam as partes relevantes do sistema.

### **Questão 8**

A recomendação seria priorizar imediatamente a criação de testes para o módulo útil, que apresenta apenas 18% de cobertura. Módulos utilitários frequentemente contêm funções de apoio usadas em todo o sistema, como formatação de dados, cálculos e validações, e bugs nessas funções podem impactar Controles e Services mesmo que estes estejam bem cobertos. A equipe deve mapear quais funções utilitárias são mais críticas para o fluxo de alertas e escrever testes unitários para elas, buscando atingir ao menos 70-80% de cobertura nesse módulo.

### **Questão 9**

Alta cobertura indica que o código foi executado pelos testes, mas não garante que os cenários corretos foram verificados. Um teste pode percorrer uma linha de código sem validar adequadamente o resultado. Além disso, cobertura não detecta problemas de integração entre módulos, falhas em condições de borda, problemas de concorrência, nem erros de lógica de negócio sutis. Portanto, é possível ter 95% de cobertura e ainda assim ter defeitos em produção por testes mal escritos.

### **Questão 10**

O que faz: o comando `npm test`. executa a suíte de testes automatizados configurada no projeto Node.js, rodando todos os arquivos de teste definidos no `package.json`.

Quando é executado: normalmente é executado automaticamente pelo pipeline CI/CD a cada push ou pull request, antes de qualquer etapa de build ou deploy.

Importância no INPE: garante que nenhuma alteração no código introduza regressões nas funcionalidades críticas do sistema, como o recebimento de denúncias de queimadas ou o disparo de alertas para a população, antes que o código chegue ao ambiente de produção.

### **Questão 11**

Não concordo com a decisão. Vulnerabilidades classificadas como High pelo npm audit representam riscos sérios de segurança, como possibilidade de execução remota de código, vazamento de dados ou negação de serviço. No contexto do INPE, onde o sistema lida com alertas de emergência para a população, uma falha de segurança poderia resultar em alertas falsos, supressão de alertas reais ou comprometimento dos dados dos usuários. O correto seria bloquear o deploy, atualizar ou substituir as dependências vulneráveis, e só liberar a implantação após a correção e nova execução do npm audit sem vulnerabilidades críticas.

### **Questão 12**

Vulnerabilidade em dependências ocorre quando uma biblioteca de terceiros utilizada pelo projeto possui uma falha de segurança conhecida que pode ser explorada. Como os projetos modernos dependem de dezenas ou centenas de pacotes externos, qualquer um deles pode conter falhas.

No projeto INPE, um exemplo concreto seria o uso de uma versão desatualizada da biblioteca axios (usada para requisições HTTP). Se essa versão contiver uma vulnerabilidade de SSRF (Server-Side Request Forgery), um atacante poderia manipular as requisições do sistema para acessar recursos internos não autorizados, comprometendo a integridade dos alertas ambientais.

### **Questão 13**

Não, 97% de cobertura não garante que o software está livre de erros. A cobertura mede quais linhas de código foram executadas, não se os resultados foram verificados corretamente. É possível ter alta cobertura com testes que não fazem assertions significativas. Além disso, os 3% não cobertos podem justamente conter um bug crítico. Erros de lógica de negócio, problemas de integração e falhas em condições de borda podem existir mesmo em código com altíssima cobertura.

### **Questão 14**

Os relatórios de cobertura permitem ao gerente identificar quais módulos do sistema estão adequadamente testados e quais representam riscos. Com essas informações, ele pode decidir de forma embasada se o sistema está suficientemente validado para ir à produção, ou se é necessário pausar o deploy e solicitar a equipe que escreva mais testes para as áreas críticas com baixa cobertura. No INPE, por exemplo, se o módulo de alertas de queimadas estiver com 20% de cobertura, o gerente tem evidência concreta para bloquear o deploy.

### **Questão 15**

Apesar de todos os testes passarem e a cobertura estar em um nível aceitável (86%), a presença de 5 vulnerabilidades classificadas como high e um bloqueador para o deploy. A recomendação é não realizar a implantação até que todas as vulnerabilidades High sejam corrigidas, seja por atualização das dependências, substituição das bibliotecas afetadas ou aplicação de patches. Vulnerabilidades High podem ser exploradas ativamente e comprometer a segurança e disponibilidade do sistema de alertas, colocando em risco tanto a infraestrutura quanto os usuários.

### **Questão 16**

Os testes automatizados são um componente fundamental da Integração Contínua (CI). Na CI, cada vez que um desenvolvedor integra código ao repositório, o pipeline é acionado automaticamente e executa os testes. Sem testes automatizados, seria impossível validar rapidamente se uma nova alteração quebrou funcionalidades existentes. Os testes são, portanto, o mecanismo que dá confiança ao processo de integração contínua permitindo que múltiplos desenvolvedores integrem código com frequência e segurança.

### **Questão 17**

A funcionalidade de recebimento de denúncias de queimadas é crítica e sensível ao tempo - um erro pode atrasar o acionamento de brigadas de combate a incêndio. Por isso, testes automatizados rigorosos são essenciais para garantir que: o formulário de denúncia valide os dados corretamente, a localização geográfica seja registrada com precisão, o alerta seja disparado para as autoridades competentes sem falhas, e o sistema se mantenha disponível sob alta carga (muitas denúncias simultâneas em períodos de seca). Qualquer regressão nessa funcionalidade deve ser detectada imediatamente no pipeline, antes de chegar à produção.

### Questão 18

Build: indica se o processo de compilação/build do projeto foi concluído com sucesso. Se estiver failing, o código não está compilando e nenhuma outra etapa deve prosseguir.

Tests: mostra se os testes automatizados estão passando. Indica que as funcionalidades implementadas estão se comportando conforme o esperado.

Coverage: exibe o percentual de cobertura de testes, indicando qual proporção do código está sendo exercitada pelos testes e sinalizando áreas que precisam de atenção.

Vulnerabilities: informa se há vulnerabilidades conhecidas nas dependências do projeto, classificadas por severidade, alertando a equipe sobre riscos de segurança.

### Questão 19

A afirmação está incorreta. A cobertura de testes e a revisão de código são práticas complementares, não excludentes. A cobertura mede apenas se o código foi executado pelos testes; a revisão de código analisa aspectos que os testes não capturam, como clareza, manutenibilidade, padrões arquiteturais, lógica de negócio mal implementada e boas práticas de segurança. Um código com 90% de cobertura pode conter testes mal escritos, lógicas duplicadas, acoplamentos desnecessários ou falhas conceituais que só uma revisão humana detectaria. Portanto, quanto maior a criticidade do sistema, como o INPE, mais importante é manter ambas as práticas.

### Questão 20

Como responsável pela aprovação do deploy do projeto INPE, analisaria os seguintes indicadores:

1. Taxa de testes passando: garante que nenhuma funcionalidade conhecida está quebrada. Qualquer teste falhando é um bloqueador imediato.
2. Cobertura de testes (mínimo recomendado: 70-80%): indica se as partes críticas do sistema estão sendo validadas. Cobertura baixa significa risco de bugs não detectados chegarem à produção.
3. Vulnerabilidades (npm audit): verificaria se há vulnerabilidades High ou Critical. Se existirem, o deploy seria bloqueado até a correção, pois o sistema lida com dados sensíveis e alertas públicos.
4. Status do Build: o build deve estar verde (passing). Build quebrado impede qualquer análise posterior.

5. Relatório de cobertura por modulo: analisaria módulos críticos individualmente para garantir que nenhum componente essencial está com cobertura inadequada, mesmo que a média geral seja aceitável.